DEPARTMENT OF
ARTIFICIAL INTELLIGENCE AND DATA SCIENCE

22IT401 -WEB TECHNOLOGY

LABORATORY RECORD

BONAFIDE CERTIFICATE

Name : .....

Register Number : .....

Academic Year : .....

Year / Semester : .....

Department : .....

This is to certify that this Laboratory Record is the bonafide work done by the above student, who has successfully completed the prescribed experiments as part of the _____ _____Laboratory during the Academic Year _____

Faculty In-charge **Head of the Department**

Submitted for the End Semester Practical Examination held on _____

Internal Examiner **External Examiner**

VISION OF THE INSTITUTION

To be a leading and path breaking Institution in multi-disciplinary education, research and industry related development for meeting the challenges of New India.

MISSION OF THE INSTITUTION

M1: Provide quality Engineering Education, Foster Research and Development; inculcate innovation in Engineering and Technology through state-of-the-art infrastructure.

M2: Nurture young men and women capable of assuming leadership roles in the society for the betterment of the country.

M3: Collaborate with industry, government organizations and society for curriculum alignment and focused, relevant outreach activities.

DEPARTMENT OF ARTIFICIAL INTELLIGENCE AND DATA SCIENCE

VISION:

To Impart Quality Education to Stimulate Academic Excellence, Research, And Skills in Students in the Areas of Data Science and Artificial Intelligence at National and Global Levels to Build an Ecosystem to Contribute Significantly to Society.

MISSION:

M1: Develop Technically Competent and Socially Responsible Professionals.

M2: Inculcate Professional Ethics, Human Values, Leadership Qualities and Lifelong Learning.

M3: To Prepare Students to Excel in The Field of Artificial Intelligence Through Collaboration with the AI Expertise Via the Centre of Excellence.

M4: Inspire Students to Develop Their Ideas into Products and Transforming Them into Aspirant Ai Professionals and Entrepreneurs.

Programme Educational Objectives (PEOs)

PEO1 - To prepare students to critically analyze existing literature in an area of specialization and enhance research to ethically develop innovative and technology-oriented methodologies with industrial support to solve the problems.

PEO2 - To provide students with strong foundational concepts advanced techniques and tools in order to enable them to build solutions for societal needs.

PEO3 – To enable graduates to pursue research and gain a successful career in academia through industrial association.

Program Outcomes (POs)

Graduates of the programme Artificial Intelligence and Data Science will be able to:

1. **Engineering Knowledge**: Apply knowledge of mathematics, natural science, computing, engineering fundamentals and an engineering specialization as specified in WK1 to WK4 respectively to develop to the solution of complex engineering problems.
2. **Problem Analysis**: Identify, formulate, review research literature and analyze complex engineering problems reaching substantiated conclusions with consideration for sustainable development. (WK1 to WK4).
3. **Design/Development of Solutions**: Design creative solutions for complex engineering problems and design/develop systems/components/processes to meet identified needs with consideration for the public health and safety, whole-life cost, net zero carbon, culture, society and environment as required. (WK5)
4. **Conduct Investigations of Complex Problems:** Conduct investigations of complex engineering problems using research-based knowledge including design of experiments, modelling, analysis & interpretation of data to provide valid conclusions. (WK8).
5. **Engineering Tool Usage:** Create, select and apply appropriate techniques, resources and modern engineering & IT tools, including prediction and modelling recognizing their limitations to solve complex engineering problems. (WK2 and WK6).
6. **The Engineer and The World**: Analyze and evaluate societal and environmental aspects while solving complex engineering problems for its impact on sustainability with reference to economy, health, safety, legal framework, culture and environment. (WK1, WK5, and WK7).
7. **Ethics**: Apply ethical principles and commit to professional ethics, human values, diversity and inclusion; adhere to national & international laws. (WK9)
8. **Individual and Collaborative Team Work**: Function effectively as an individual, and as a member or leader in diverse/multi-disciplinary teams.
9. **Communication**: Communicate effectively and inclusively within the engineering community and society at large, such as being able to comprehend and write effective reports and design documentation, make effective presentations considering cultural, language, and learning differences.
10. **Project Management and Finance**: Apply knowledge and understanding of engineering management principles and economic decision-making and apply these to one's own work, as a member and leader in a team, and to manage projects and in multidisciplinary environments.
11. **Lifelong Learning**: Recognize the need for, and have the preparation and ability for
    i) independent and life-long learning ii) adaptability to new and emerging technologies and
    ii) critical thinking in the broadest context of technological change. (WK8)

Program Specific Outcomes (PSOs)

Graduates of the programme Artificial Intelligence and Data Science will be able to

1. Adapt Information and Communication Technologies (ICT) and foundational concepts of Electronics and Communication Engineering to analyze, design and develop solutions.
2. Provide knowledge to create quality products for scientific and business applications by incorporating novel ideas and best practices.

DO'S & DONT'S FOR LABORATORY

DO'S

- Wear the proper uniform and ID card while attending all lab sessions.
- Maintain discipline in the lab and follow the instructions given by the faculty or lab instructor.
- Handle computers, peripherals, and network equipment carefully.
- Complete all assigned batch work on time and cooperate with your batch members.
- Use the systems only for academic purposes like coding, assignments, and project work

DONT'S

- Do not install, uninstall, or modify any software or system settings without permission.
- Do not browse social media, entertainment platforms, or access unauthorized websites.
- Do not bring food, drinks, or snacks inside the laboratory.
- Do not misuse or mishandle keyboards, mice, cables, or other peripherals.
- Do not leave the system without logging out or leave the workspace untidy after use.

SAFETY PRECAUTIONS FOR WORKING IN THE LABORATORY

- Keep your workspace clean and free from unnecessary items to avoid accidents.
- Report any damaged wires, loose connections, or malfunctioning equipment to the lab instructor immediately.
- Follow all safety instructions displayed in the laboratory and given by faculty.
- Switch off power supply and function generator while working on the circuit.
- Handle computers, electrical equipment, and cables with care to prevent shocks or damage

LIST OF EXPERIMENTS

S. NO	NAME OF THE EXPERIMENTS
1.	Create a webpage with the following using HTML. To embed an image map in a webpage. To fix the hot spots. Show all the related information when the hotspots are clicked.
2.	Create a webpage with all types of Cascading style sheets
3.	Client Side Scripts for Validating Web Form Controls using DHTML.
4.	Installation of Apache Tomcat web server.
5.	Write programs in Java using Servlets:
	i. To invoke servlets from HTML forms
	ii. Session Tracking
6.	Write programs in Java to create three-tier applications using JSP and Databases
	(i) For conducting on-line examination
	(ii) For displaying student mark list. Assume that student information is available in a database which has been stored in a database server
7.	XML Schema for Student Detail

List of Experiments Mapping with COs, POs &PSOs

Exp. No.	Name of the Experiment	COs	POs	PSOs
1	Create a webpage with the following using HTML. To embed an image map in a webpage. To fix the hot spots. Show all the related information when the hotspots are clicked.	CO1 , CO5	PO1, PO2, PO5	PSO2
2	Create a webpage with all types of Cascading style sheets	CO1, CO5	PO1, PO2, PO5	PSO2
3	Client Side Scripts for Validating Web Form Controls using DHTML.	CO2, CO5	PO1, PO2, PO5	PSO2

4	Installation of Apache Tomcat web server.	CO3 , CO5	PO1, PO5	PSO2
5	Write programs in Java using Servlets:	CO3, CO5	PO1, PO2, PO3, PO5	PSO1, PSO2
6	(i)To invoke servlets from HTML forms	CO3, CO5	PO1, PO2, PO3, PO5	PSO1, PSO2
7	(ii) Session Tracking	CO4, CO5	PO1, PO2, PO5	PSO1, PSO2

LIST OF EXPERIMENTS

S.No.	Name of the Experiment	Date	Page No	Marks	Sign

AVERAGE =

Exp 1 **CREATING A WEB PAGE USING IMAGE MAP**

DATE :

AIM:

To create a webpage using HTML that embeds an image map of India and defines hotspots for southern states. When the user clicks on each hotspot, the browser should open a detailed page with related information for that state.

ALGORITHM:

Step 1: Open Visual Studio Code and create a new HTML file named index.html. Insert the India map using the <img> tag and define an image map using the <map> and <area> tags.

Step 2: In the <map> tag, create hotspots using <area> with proper shape, coords, and href attributes to link to individual state pages.

Step 3: Create separate HTML files like kerala.html, tamilnadu.html, etc., and add relevant content such as capital, language, and tourist places for each state.

Step 4: Run index.html in a browser (or using Live Server in VS Code) and click on the hotspots to test navigation.

Step 5: Verify that each hotspot opens the correct state page with neat and complete information.

SOURCE CODE:

ndex.html

```
<!DOCTYPE html>
<html>
<head>
  <title>Explore Southern States of India</title>
</head>
<body bgcolor="#f9f7dc">

  <h1 style="text-align:center; color:darkgreen;"><u>Explore Southern States of India</u></h1>

  <p style="width:90%; margin:auto; font-size:18px;">
    India is a diverse country with 28 states and 8 Union Territories. Below is a map of India
    highlighting southern states. Click on each hotspot (state) in the map to view more details.
  </p>

  <center>

    <img
      src="https://upload.wikimedia.org/wikipedia/commons/2/26/Map_of_India.png"
```

```
        alt="Map of India"
        width="450"
        height="512"
        usemap="#indiastates">

    <!-- Image Map with working hotspot coordinates -->
    <map name="indiastates">
        <area shape="circle" coords="160,410,12" href="tamilnadu.html" alt="Tamil Nadu">
        <area shape="circle" coords="140,400,10" href="kerala.html" alt="Kerala">
        <area shape="circle" coords="130,360,12" href="karnataka.html" alt="Karnataka">
        <area shape="circle" coords="170,340,12" href="andhrapradesh.html" alt="Andhra Pradesh">
    </map>
  </center>

  <h3 style="text-align:center;">Click on any southern state to learn more about it!</h3>

</body>
</html>
```

Tamilnadu.html

```
<!DOCTYPE html>
<html>
<head>
  <title>Tamil Nadu</title>
</head>
<body bgcolor="#ffe6e6">
  <h1 style="color:maroon;">Welcome to Tamil Nadu</h1>

  <!-- Added image for Tamil Nadu -->
  <center>
    <img src="https://i.pinimg.com/564x/4e/e9/c0/4ee9c0fbdfe8d8c4a0d3f862bcff37d9.jpg"
       alt="Tamil Nadu Map"
       width="400">
  </center>

  <p><b>Capital:</b> Chennai</p>
  <p><b>Language:</b> Tamil</p>
  <p><b>Famous for:</b> Temples, Classical Dance (Bharatanatyam), Marina Beach</p>
  <p><b>Tourist Attractions:</b> Madurai, Rameswaram, Kanyakumari, Ooty</p>
  <p><a href="index.html">← Back to Map</a></p>
</body>
</html>
```

Kerala.html

```
<!DOCTYPE html>
<html>
```

```
<head>
  <title>Kerala</title>
</head>
<body bgcolor="#e6fff2">
  <h1 style="color:darkgreen;">Welcome to Kerala</h1>

  <!-- Added image for Kerala -->
  <center>
     <img src="https://i.pinimg.com/564x/0f/ba/9b/0fba9bef8d631d81f7b044aa66fda18e.jpg"
          alt="Kerala Map"
          width="400">
  </center>

  <p><b>Capital:</b> Thiruvananthapuram</p>
  <p><b>Language:</b> Malayalam</p>
  <p><b>Famous for:</b> Backwaters, Ayurveda, Coconut Trees</p>
  <p><b>Tourist Attractions:</b> Alleppey, Munnar, Wayanad, Kovalam</p>
  <p><a href="index.html">← Back to Map</a></p>
</body>
</html>
```

Karnataka.html

```
<!DOCTYPE html>
<html>
<head>
  <title>Karnataka</title>
</head>
<body bgcolor="#e6f0ff">
  <h1 style="color:navy;">Welcome to Karnataka</h1>

  <!-- Added image for Karnataka -->
  <center>
     <img src="https://i.pinimg.com/564x/78/94/d3/7894d3e65a5e48b14fb1df2ccef6a0a9.jpg"
          alt="Karnataka Map"
          width="400">
  </center>

  <p><b>Capital:</b> Bengaluru</p>
  <p><b>Language:</b> Kannada</p>
  <p><b>Famous for:</b> IT Industry, Heritage Sites, Coffee Plantations</p>
  <p><b>Tourist Attractions:</b> Mysore, Hampi, Coorg, Gokarna</p>
  <p><a href="index.html">← Back to Map</a></p>
</body>
</html>
```

Andra pradesh.html

```
<!DOCTYPE html>
<html>
<head>
  <title>Andhra Pradesh</title>
</head>
<body bgcolor="#fff5e6">
  <h1 style="color:orange;">Welcome to Andhra Pradesh</h1>

  <!-- Added image for Andhra Pradesh -->
  <center>
     <img src="https://www.shutterstock.com/image-vector/administrative-political-map-state-andhra-
260nw-2287050403.jpg"
          alt="Andhra Pradesh Map"
          width="400">
  </center>

  <p><b>Capital:</b> Amaravati</p>
  <p><b>Language:</b> Telugu</p>
  <p><b>Famous for:</b> Tirupati Temple, Kuchipudi Dance, Handicrafts</p>
  <p><b>Tourist Attractions:</b> Vijayawada, Visakhapatnam, Araku Valley</p>
  <p><a href="index.html">← Back to Map</a></p>
</body>
</html>
```

Outputs :

Tamilnadu.html

Welcome to Tamil Nadu

Capital: Chennai

Language: Tamil

Famous for: Temples, Classical Dance (Bharatanatyam), Marina Beach

Tourist Attractions: Madurai, Rameswaram, Kanyakumari, Ooty

← Back to Map

Kerala.html

Welcome to Kerala

Capital: Thiruvananthapuram

Language: Malayalam

Famous for: Backwaters, Ayurveda, Coconut Trees

Tourist Attractions: Alleppey, Munnar, Wayanad, Kovalam

← Back to Map

Karnataka.html

Welcome to Karnataka

Capital: Bengaluru
Language: Kannada
Famous for: IT Industry, Heritage Sites, Coffee Plantations
Tourist Attractions: Mysore, Hampi, Coorg, Gokarna
← Back to Map

Andra Pradesh.html

Welcome to Andhra Pradesh

Capital: Amaravati

Language: Telugu

Famous for: Tirupati Temple, Kuchipudi Dance, Handicrafts

Tourist Attractions: Vijayawada, Visakhapatnam, Araku Valley

← Back to Map

RESULT :

The webpage with an embedded image map was successfully created. Hotspots were accurately defined, and when clicked, each hotspot displayed the correct related information. The experiment achieved the intended interactive functionality.

Exp 2 CREATING A WEBPAGE WITH CASCADING STYLE SHEET

DATE :

AIM

To create a webpage using all types of Cascading Style Sheets (CSS) – inline, internal, and external – and observe how styles are applied and overridden.

ALGORITHM

Step 1: Create a folder for the experiment.

Step 2: Create two files: index.html and style.css.

Step 3: In style.css, write external CSS rules for body, headings, and paragraphs.

Step 4: In index.html, link style.css using the <link> tag in the <head> section.

Step 5: Add internal CSS using the <style> tag in the <head> section.

Step 6: Apply inline CSS to selected elements using the style attribute.

Step 7: Save the files, open index.html in a browser, and verify that all three types of CSS are applied.

SOURCE CODE

```
<!DOCTYPE html>
<html lang="en">
<head>
   <meta charset="UTF-8">
   <title>Types of Cascading Style Sheets</title>

   <!-- External CSS -->
   <link rel="stylesheet" type="text/css" href="style.css">

   <!-- Internal CSS -->
   <style>
      h2 {
         color: darkblue;
         text-align: center;
         text-decoration: underline;
      }

      .content-box {
         border: 2px solid #333;
         padding: 15px;
```

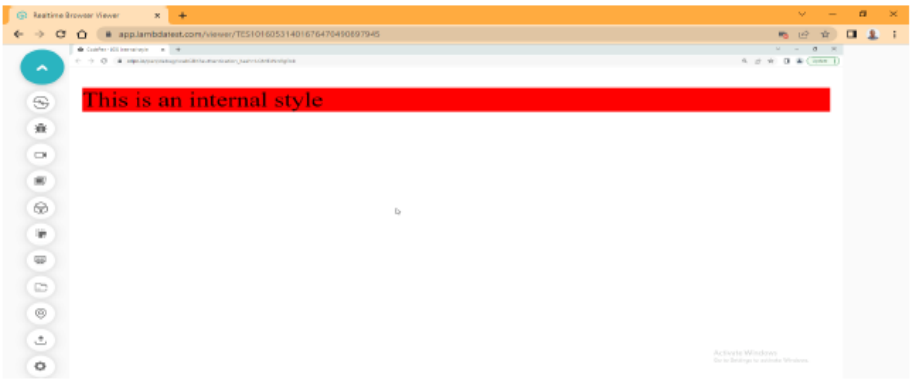
```
        margin: 20px auto;
        width: 80%;
        background-color: #f9f9f9;
    }

    .internal-text {
        font-style: italic;
    }
  </style>
</head>

<body>
  <!-- External CSS -->
  <h1>Demonstration of Cascading Style Sheets</h1>

  <!-- Internal CSS -->
  <h2>Inline, Internal and External CSS</h2>

  <div class="content-box">

    <!-- Internal + External CSS -->
    <p class="internal-text">
        This paragraph is styled using internal and external CSS.
    </p>

    <!-- Inline CSS -->
    <p style="color: red; font-weight: bold; font-size: 18px;">
        This paragraph uses <strong>inline CSS</strong>, which has the highest priority.
    </p>

    <!-- Only External CSS -->
    <p>
        This paragraph is styled only by the external stylesheet.
    </p>
  </div>

  <footer>
    <p>&copy; 2025 Web Technology Lab</p>
  </footer>
</body>
</html>
```

Style.CSS

```
/* External CSS for the webpage */
```

```
body {
    font-family: Arial, Helvetica, sans-serif;
    background-color: #eef2f5;
    margin: 0;
    padding: 0;
}

h1 {
    text-align: center;
    color: #006666;
    padding: 20px 0;
    background-color: #d0e9e9;
    margin: 0;
}

p {
    color: #333333;
    line-height: 1.6;
}

footer {
    text-align: center;
    padding: 10px 0;
    margin-top: 20px;
    background-color: #d0e9e9;
    font-size: 14px;
}
```

Outputs:



RESULT

The webpage was successfully styled using inline, internal and external CSS. All three styles were applied correctly and displayed as expected in the browser.

EX.NO: 3 **CLIENT SIDE SCRIPTS FOR VALIDATING WEB FORM**
DATE : **CONTROL USING DHTML**

AIM
To design and validate a web form using **client-side scripting** in DHTML (HTML + CSS + JavaScript).

ALGORITHM
Step 1: Create an HTML form with fields such as Name, Email, Password and Age.
Step 2: Add elements near each input to display validation messages.
Step 3: Write CSS to format the form and error messages.
Step 4: Write a JavaScript function validateForm() to validate:
- ✓ Empty fields
- ✓ Email format
- ✓ Password length
- ✓ Password confirmation
- ✓ Valid age

Step 5: Display error messages dynamically using DHTML (innerHTML).
Step 6: Allow submission only when all fields are correctly validated.

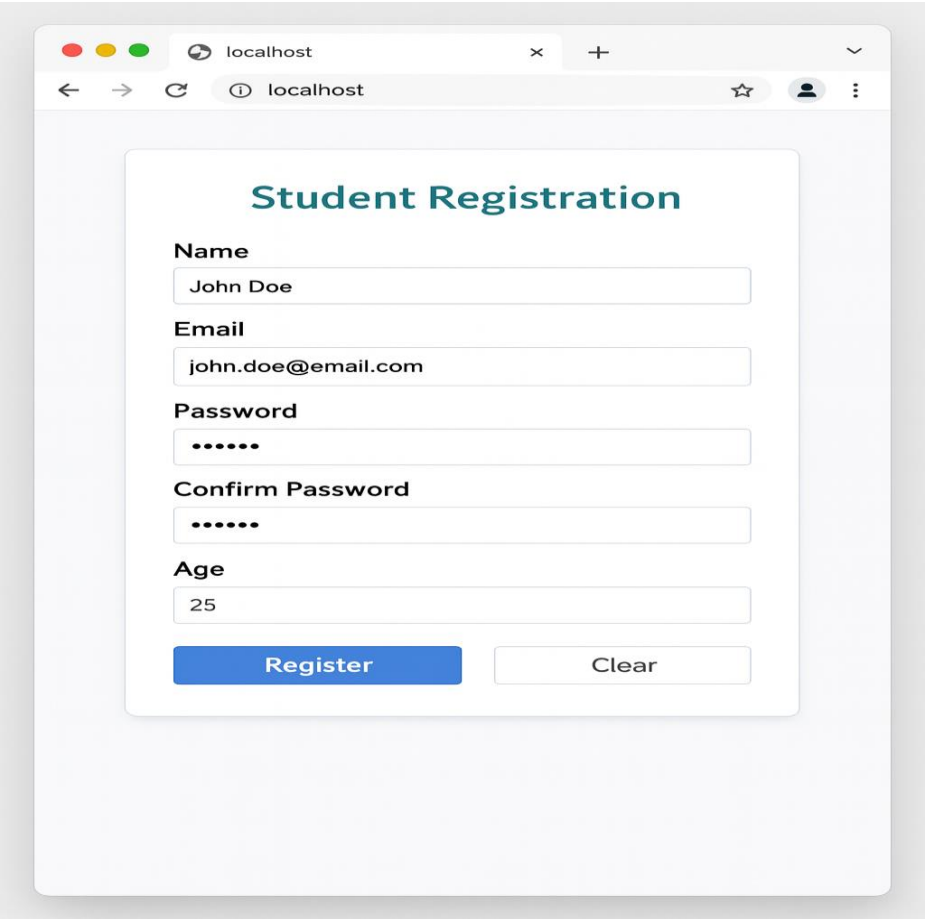
SOURCE CODE

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <title>DHTML Form Validation</title>

  <style>
    body { font-family: Arial; background:#eef2f5; }
    .form-box {
      width: 380px; background:#fff; padding:20px;
      margin:40px auto; border-radius:6px;
      box-shadow:0 0 6px rgba(0,0,0,0.2);
    }
    h2 { text-align:center; color:#006666; margin-top:0; }
    label { font-weight:bold; display:block; margin-top:10px; }
    input { width:100%; padding:6px; margin-top:5px; }
    .error { color:red; font-size:12px; }
    .btn { margin-top:15px; text-align:center; }
  </style>

  <script>
    function validateForm() {
```

```
        let valid = true;

        let name  = document.getElementById("name").value.trim();
        let email = document.getElementById("email").value.trim();
        let pass  = document.getElementById("pass").value;
        let cpass = document.getElementById("cpass").value;
        let age   = document.getElementById("age").value;

        document.querySelectorAll(".error").forEach(e => e.innerHTML = "");

        if(name === ""){
            document.getElementById("nameErr").innerHTML = "Enter your name";
            valid = false;
        }

        let pattern = /^[^@\s]+@[^@\s]+\.[^@\s]+$/;
        if(email === "" || !pattern.test(email)){
            document.getElementById("emailErr").innerHTML = "Enter a valid email";
            valid = false;
        }

        if(pass.length < 6){
            document.getElementById("passErr").innerHTML = "Min 6 characters";
            valid = false;
        }

        if(pass !== cpass){
            document.getElementById("cpassErr").innerHTML = "Passwords do not match";
            valid = false;
        }

        if(age <= 0){
            document.getElementById("ageErr").innerHTML = "Enter valid age";
            valid = false;
        }

        if(!valid) return false;

        alert("Form submitted successfully!");
        return true;
    }
  </script>
</head>
```

```
<body>
  <div class="form-box">
    <h2>Student Registration</h2>

    <form onsubmit="return validateForm();">

      <label>Name</label>
      <input type="text" id="name">
      <span id="nameErr" class="error"></span>

      <label>Email</label>
      <input type="text" id="email">
      <span id="emailErr" class="error"></span>

      <label>Password</label>
      <input type="password" id="pass">
      <span id="passErr" class="error"></span>

      <label>Confirm Password</label>
      <input type="password" id="cpass">
      <span id="cpassErr" class="error"></span>

      <label>Age</label>
      <input type="number" id="age">
      <span id="ageErr" class="error"></span>

      <div class="btn">
        <input type="submit" value="Register">
        <input type="reset" value="Clear">
      </div>
    </form>
  </div>
</body>
</html>
```

Output:



RESULT :

The client-side validation of web form controls using DHTML was successfully implemented. All input fields were validated in the browser, appropriate error messages were displayed for incorrect entries and the form permitted submission only when all data met the required criteria.

EX.NO: 4 **INSTALLING & CONFIGURING TOMCAT – WEBSERVER**

DATE :

AIM

To install, configure, and run the Apache Tomcat Web Server and verify its successful deployment for hosting Java-based web applications.

Step 1: Open a web browser and go to the official Apache Tomcat website.

Step 2: Download the Windows Installer (.exe) for Apache Tomcat.

Step 3: Locate the downloaded installer file and double-click to start the setup.

Step 4: When the Setup Wizard opens, click the **Next** button.

Step 5: Read and accept the License Agreement, then click **Next**.

Step 6: Select the required components (Tomcat Manager, Host Manager, Docs, Examples).

Step 7: Choose the installation directory for Tomcat.

Step 8: Configure the Server Port (default: 8080).

Step 9: Enter the **Administrator Username and Password** for Tomcat Manager.

Step 10: Select the Java (JDK/JRE) installation folder when prompted.

Step 11: Click **Install** to begin the installation process.

Step 12: Wait for the installer to copy files and configure the webserver.

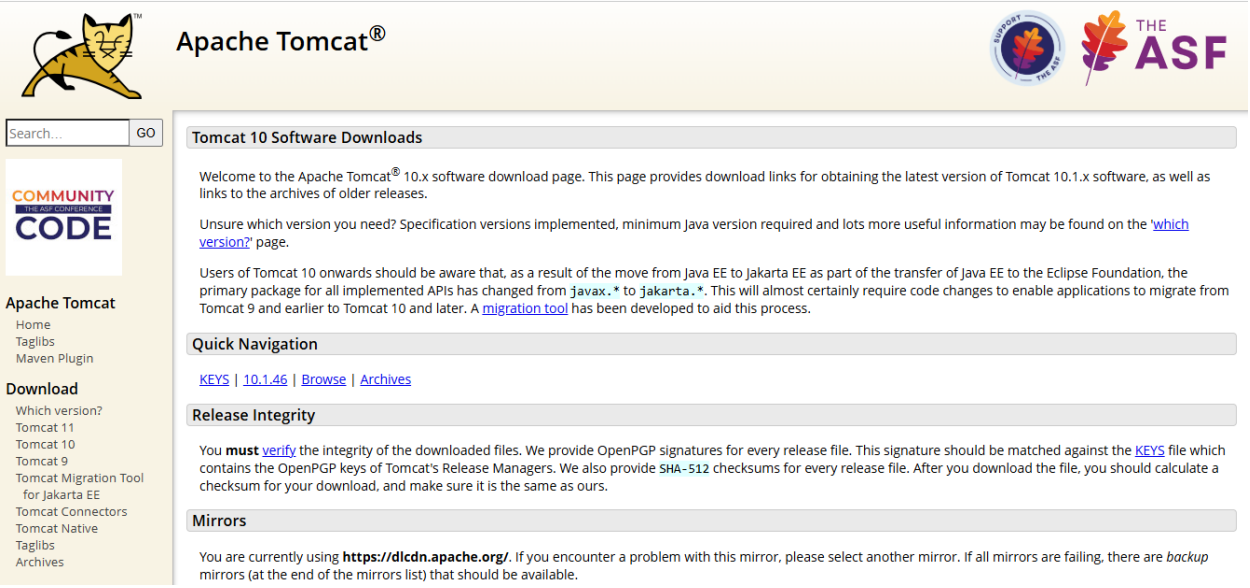
Step 13: After successful installation, click **Finish**.

Step 14: Open a browser and type http://localhost:8080/.

Step 15: Verify that the Apache Tomcat Homepage is displayed.

PROCEDURE

Procedure 1 : Open the Official Apache Tomcat Website



Apache Tomcat®

Search... GO

COMMUNITY CODE

Apache Tomcat
- Home
- Taglibs
- Maven Plugin

Download
- Which version?
- Tomcat 11
- Tomcat 10
- Tomcat 9
- Tomcat Migration Tool for Jakarta EE
- Tomcat Connectors
- Tomcat Native
- Taglibs
- Archives

Tomcat 10 Software Downloads

Welcome to the Apache Tomcat® 10.x software download page. This page provides download links for obtaining the latest version of Tomcat 10.1.x software, as well as links to the archives of older releases.

Unsure which version you need? Specification versions implemented, minimum Java version required and lots more useful information may be found on the 'which version?' page.

Users of Tomcat 10 onwards should be aware that, as a result of the move from Java EE to Jakarta EE as part of the transfer of Java EE to the Eclipse Foundation, the primary package for all implemented APIs has changed from `javax.*` to `jakarta.*`. This will almost certainly require code changes to enable applications to migrate from Tomcat 9 and earlier to Tomcat 10 and later. A migration tool has been developed to aid this process.

Quick Navigation

KEYS | 10.1.46 | Browse | Archives

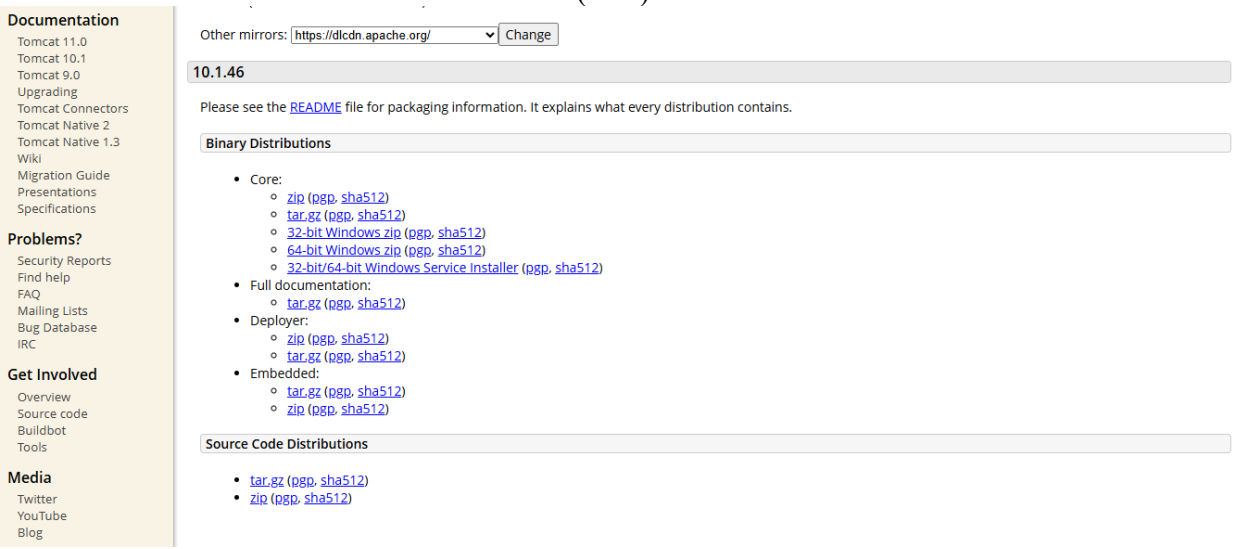
Release Integrity

You **must** verify the integrity of the downloaded files. We provide OpenPGP signatures for every release file. This signature should be matched against the KEYS file which contains the OpenPGP keys of Tomcat's Release Managers. We also provide `SHA-512` checksums for every release file. After you download the file, you should calculate a checksum for your download, and make sure it is the same as ours.

Mirrors

You are currently using **https://dlcdn.apache.org/**. If you encounter a problem with this mirror, please select another mirror. If all mirrors are failing, there are *backup* mirrors (at the end of the mirrors list) that should be available.

Procedure 2 — Select the Windows Installer (.exe) Download Link



Documentation
- Tomcat 11.0
- Tomcat 10.1
- Tomcat 9.0
- Upgrading
- Tomcat Connectors
- Tomcat Native 2
- Tomcat Native 1.3
- Wiki
- Migration Guide
- Presentations
- Specifications

Problems?
- Security Reports
- Find help
- FAQ
- Mailing Lists
- Bug Database
- IRC

Get Involved
- Overview
- Source code
- Buildbot
- Tools

Media
- Twitter
- YouTube
- Blog

Other mirrors: https://dlcdn.apache.org/ Change

10.1.46

Please see the README file for packaging information. It explains what every distribution contains.

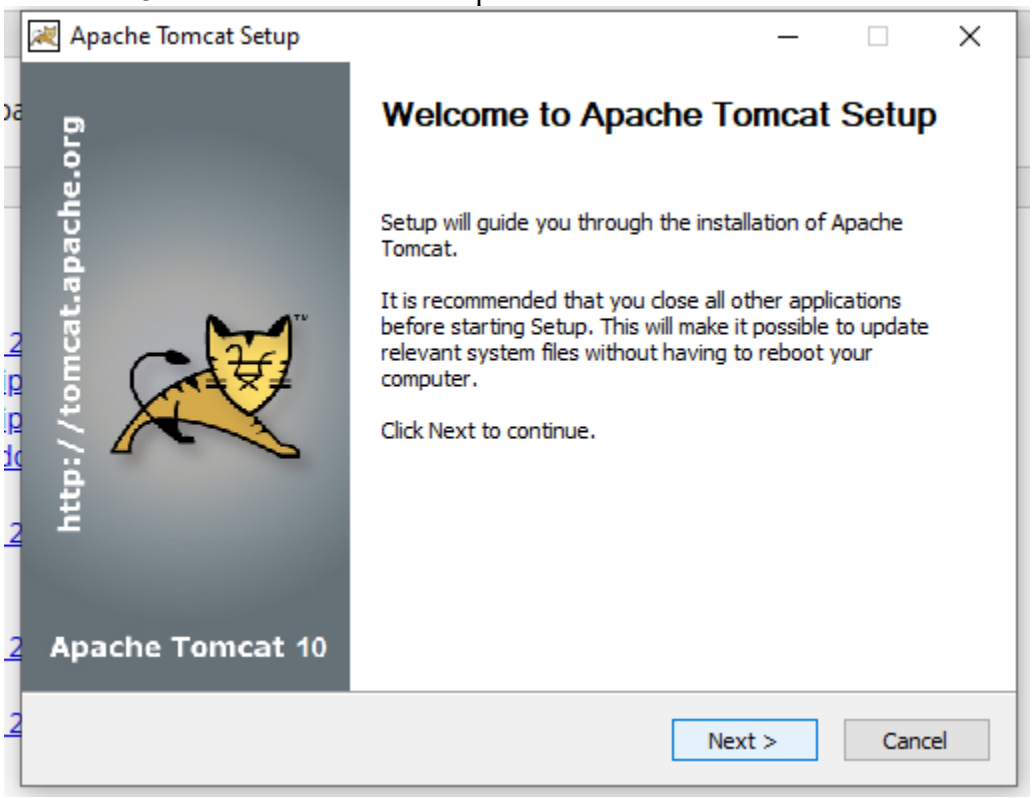
Binary Distributions

- Core:
  - zip (pgp, sha512)
  - tar.gz (pgp, sha512)
  - 32-bit Windows zip (pgp, sha512)
  - 64-bit Windows zip (pgp, sha512)
  - 32-bit/64-bit Windows Service Installer (pgp, sha512)
- Full documentation:
  - tar.gz (pgp, sha512)
- Deployer:
  - zip (pgp, sha512)
  - tar.gz (pgp, sha512)
- Embedded:
  - tar.gz (pgp, sha512)
  - zip (pgp, sha512)

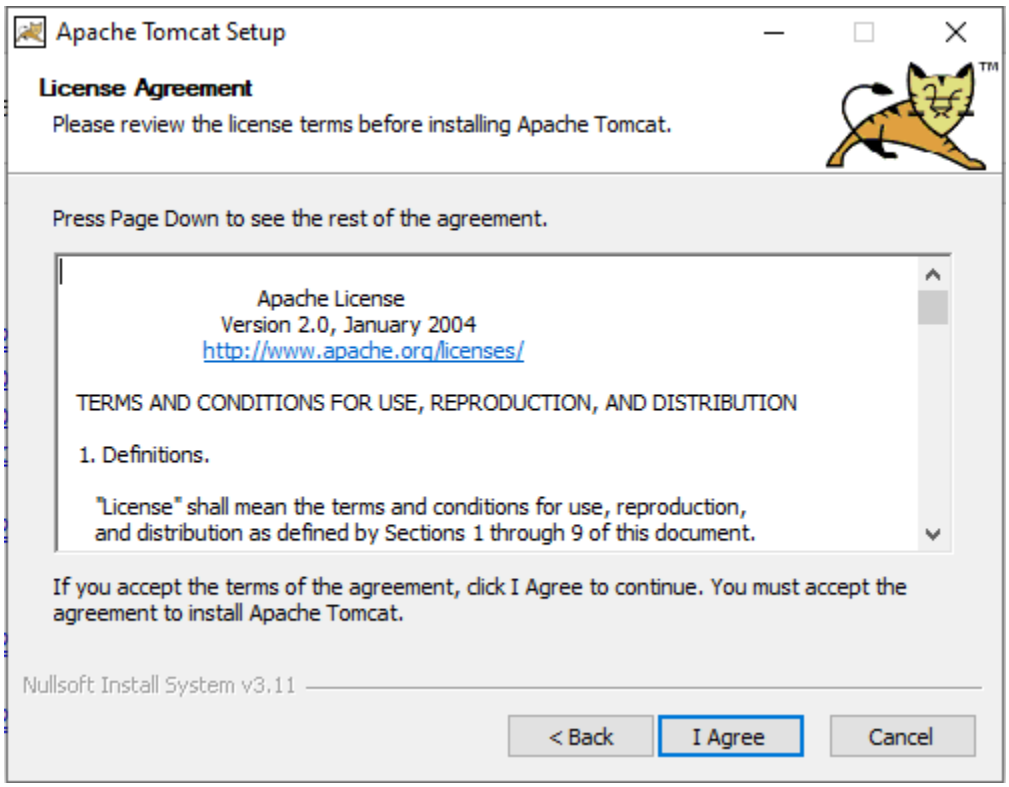
Source Code Distributions

- tar.gz (pgp, sha512)
- zip (pgp, sha512)

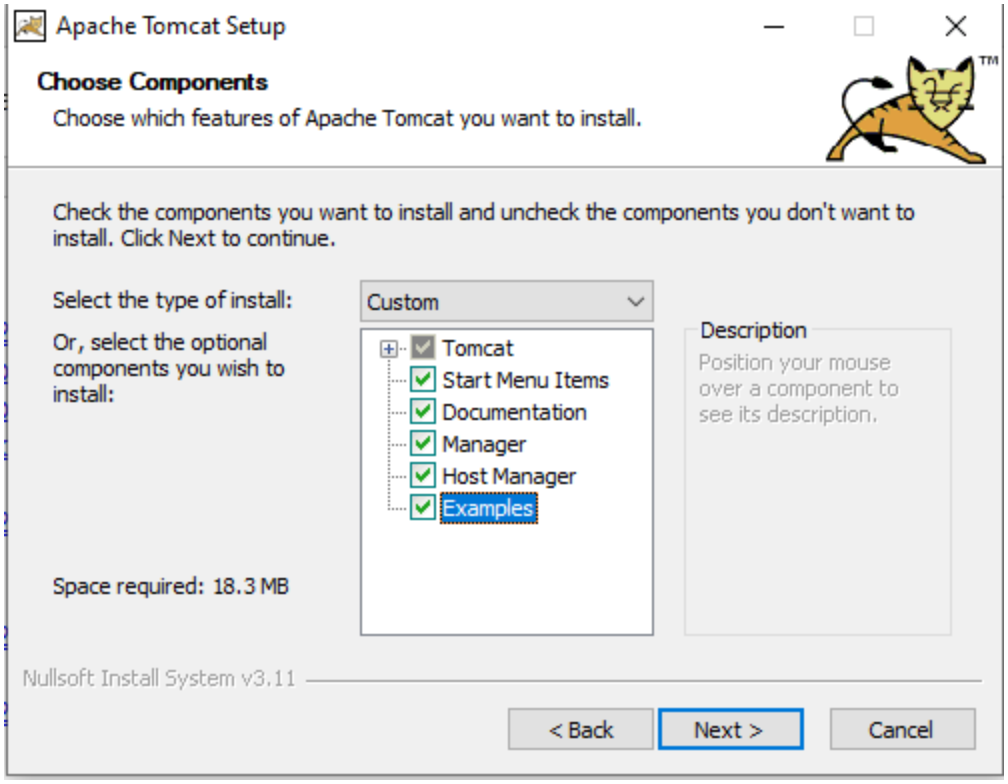
Procedure 3 — Tomcat Installer Setup Wizard – Click NEXT



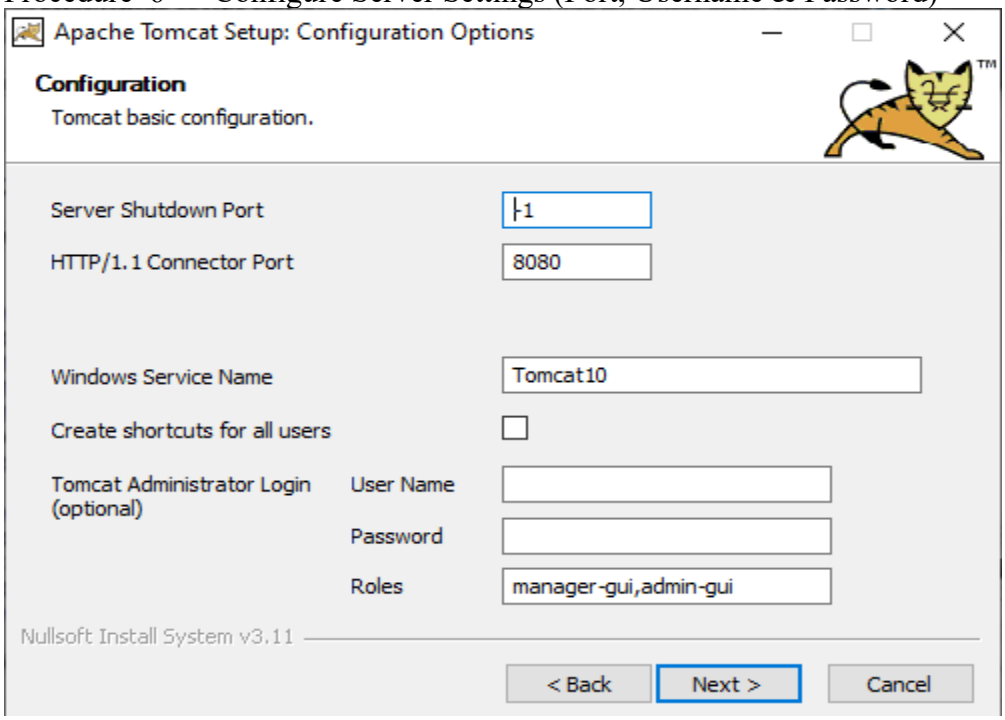
Procedure 4 — Accept the License Agreement



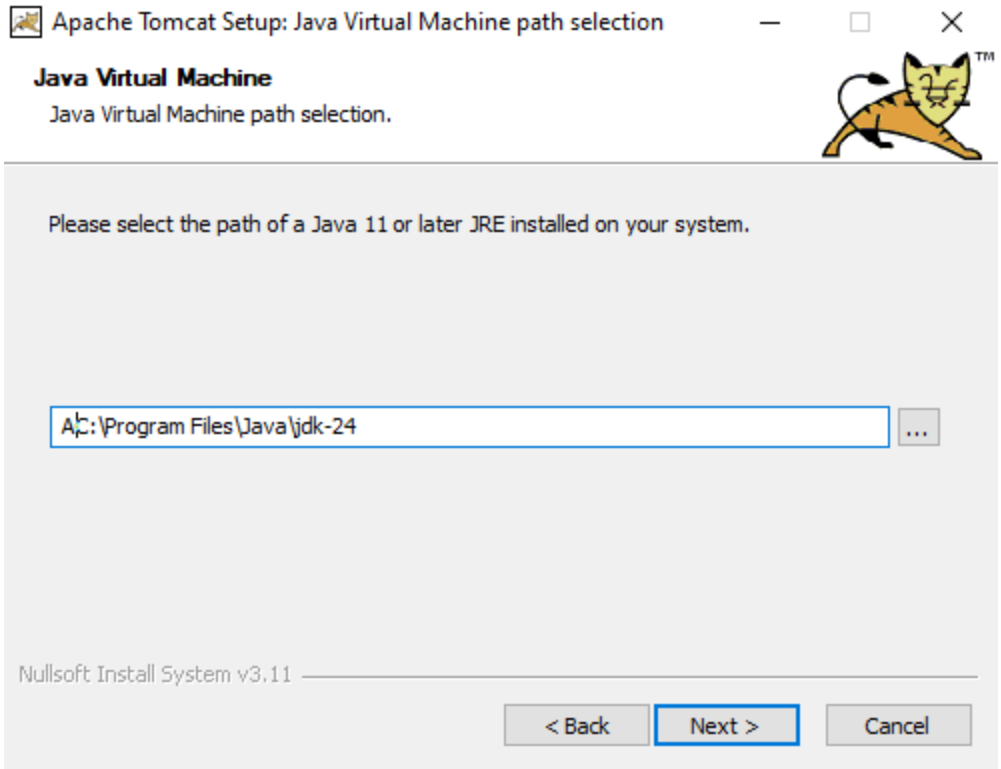
Procedure 5 — Select Tomcat Components and Click NEXT



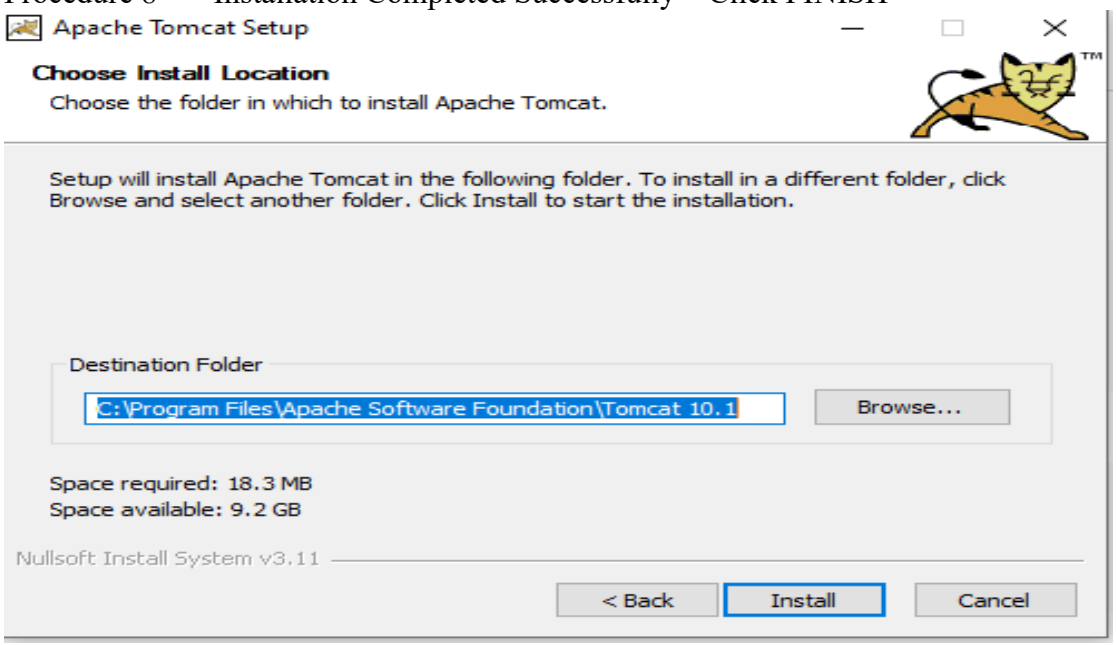
Procedure 6 — Configure Server Settings (Port, Username & Password)

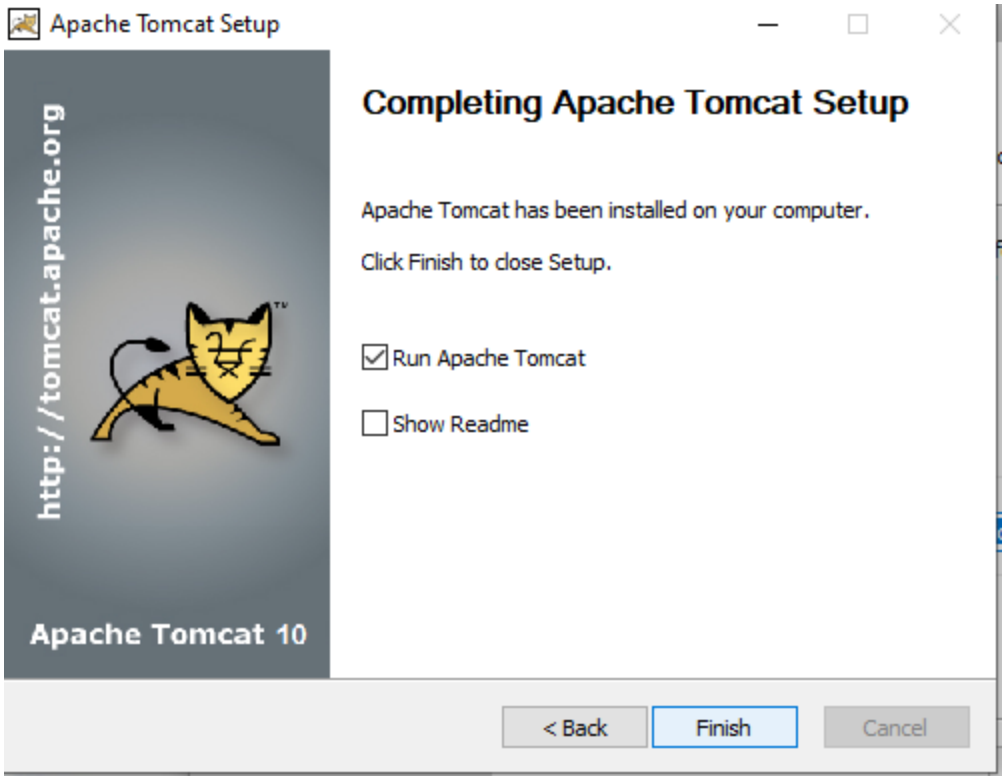


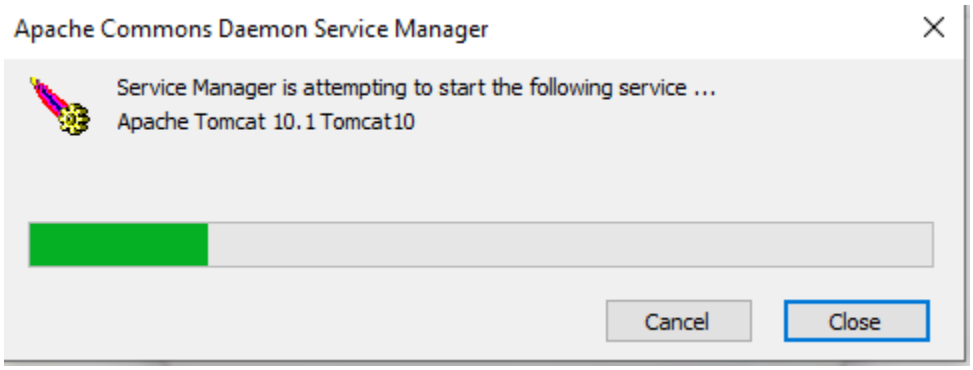
Procedure 7 — Select Java (JRE/JDK) Folder



Procedure 8 — Installation Completed Successfully – Click FINISH







Finally Open Browser and Access http://localhost:8080

RESULT :

The Apache Tomcat Webserver was successfully installed and configured. The installation steps were completed as per the procedure.

EXP 5 i) **INVOKING SERVLET FROM HTML FORMS**

DATE :

AIM

To develop an HTML form and a Java Servlet that is invoked from the form and displays the submitted data.

ALGORITHM

Step 1: Start a new Dynamic Web Project in the IDE (e.g., Eclipse) with Servlet support enabled.

Step 2: Create an HTML file studentForm.html in the WebContent (or webapp) folder.

Step 3: In the HTML file, design a form with fields such as *Name*, *Register Number* and *Department*.

Step 4: Set the action attribute of the form to the servlet URL pattern (for example, StudentServlet) and the method as POST.

Step 5: Create a Java servlet class StudentServlet that extends HttpServlet.

Step 6: Use the @WebServlet("/StudentServlet") annotation (or configure in web.xml) to map the servlet to the given URL pattern.

Step 7: Override the doPost() method in the servlet.

Step 8: Inside doPost(), read the form values using request.getParameter("parameterName").

Step 9: Obtain a PrintWriter object using response.getWriter() and set the content type to "text/html".

Step 10: Display the form data in a formatted HTML response using the PrintWriter.

Step 11: Save all files, deploy the application on the server (e.g., Apache Tomcat), and run the HTML form.

Step 12: Enter sample values in the form and submit to verify that the servlet is invoked and the data is displayed.

SOURCE CODE

HTML Form – studentForm.html

```
<!DOCTYPE html>
<html>
<head>
    <meta charset="UTF-8">
    <title>Student Information Form</title>
</head>
<body>
    <h2>Student Information Form</h2>
    <form action="StudentServlet" method="post">
        <table>
            <tr>
                <td>Name</td>
                <td><input type="text" name="name" required></td>
            </tr>
            <tr>
                <td>Register Number</td>
                <td><input type="text" name="regno" required></td>
            </tr>
            <tr>
                <td>Department</td>
                <td><input type="text" name="dept" required></td>
            </tr>
            <tr>
                <td colspan="2" style="text-align:center;">
                    <input type="submit" value="Submit">
                    <input type="reset" value="Clear">
                </td>
            </tr>
        </table>
    </form>
</body>
</html>
```

Servlet – StudentServlet.java

```
import java.io.IOException;
import java.io.PrintWriter;
import javax.servlet.ServletException;
import javax.servlet.annotation.WebServlet;
import javax.servlet.http.HttpServlet;
import javax.servlet.http.HttpServletRequest;
import javax.servlet.http.HttpServletResponse;

@WebServlet("/StudentServlet")
public class StudentServlet extends HttpServlet {

    private static final long serialVersionUID = 1L;

    @Override
    protected void doPost(HttpServletRequest request,
                          HttpServletResponse response)
            throws ServletException, IOException {

        // Read form data
        String name = request.getParameter("name");
        String regno = request.getParameter("regno");
        String dept = request.getParameter("dept");

        // Set response type
        response.setContentType("text/html");

        // Get writer to send response
        PrintWriter out = response.getWriter();

        out.println("<!DOCTYPE html>");
        out.println("<html>");
        out.println("<head>");
        out.println("<meta charset='UTF-8'>");
```

```
        out.println("<title>Student Details</title>");
        out.println("</head>");
        out.println("<body>");
        out.println("<h2>Submitted Student Details</h2>");
        out.println("<table border='1' cellpadding='5' cellspacing='0'>");
        out.println("<tr><th>Field</th><th>Value</th></tr>");
        out.println("<tr><td>Name</td><td>" + name + "</td></tr>");
        out.println("<tr><td>Register Number</td><td>" + regno + "</td></tr>");
        out.println("<tr><td>Department</td><td>" + dept + "</td></tr>");
        out.println("</table>");
        out.println("</body>");
        out.println("</html>");

        out.close();
    }
}
```

Output

Login Page (login.html)

After Login (LoginServlet)

RESULT

Thus, a servlet was successfully invoked from an HTML form, and the submitted student information was displayed in the browser using the servlet.

EXP 5 ii) **SESSION TRACKING USING SERVLETS**

DATE:

AIM

To develop a Java Servlet application that demonstrates session tracking using Http Session to maintain user information across multiple requests.

ALGORITHM

Step 1: Create a new Dynamic Web Project with Servlet support.

Step 2: Create an HTML file login.html that accepts a user name from the user.

Step 3: Set the form action to LoginServlet and the method as POST.

Step 4: Create a servlet class LoginServlet extending HttpServlet.

Step 5: In LoginServlet, override the doPost() method.

Step 6: Read the user name from the request using request.getParameter("username").

Step 7: Create or retrieve the session object using request.getSession().

Step 8: Store the user name in the session using session.setAttribute("username", userName).

Step 9: Send an HTML response or redirect/forward the request to another servlet (WelcomeServlet).

Step 10: Create another servlet WelcomeServlet that reads the session attribute using session.getAttribute("username").

Step 11: Display a welcome message using the user name from the session and optionally display the session ID.

Step 12: Deploy the project on the server, open login.html, submit the form multiple times or open WelcomeServlet directly to observe that the same session data is reused until the session expires or is invalidated.

SOURCE CODE

(a) HTML Form – login.html

```
<!DOCTYPE html>
<html>
```

```
<head>
    <meta charset="UTF-8">
    <title>Session Tracking - Login</title>
</head>
<body>
    <h2>Login Page</h2>
    <form action="LoginServlet" method="post">
        <table>
            <tr>
                <td>User Name</td>
                <td><input type="text" name="username" required></td>
            </tr>
            <tr>
                <td colspan="2" style="text-align:center;">
                    <input type="submit" value="Login">
                    <input type="reset" value="Clear">
                </td>
            </tr>
        </table>
    </form>
</body>
</html>
```

(b) Servlet – LoginServlet.java

```
import java.io.IOException;
import java.io.PrintWriter;
import javax.servlet.ServletException;
import javax.servlet.annotation.WebServlet;
import javax.servlet.http.HttpServlet;
import javax.servlet.http.HttpServletRequest;
import javax.servlet.http.HttpServletResponse;
import javax.servlet.http.HttpSession;

@WebServlet("/LoginServlet")
public class LoginServlet extends HttpServlet {

    private static final long serialVersionUID = 1L;
```

```
    @Override
    protected void doPost(HttpServletRequest request,
                          HttpServletResponse response)
            throws ServletException, IOException {

        String userName = request.getParameter("username");

        // Create or get existing session
        HttpSession session = request.getSession();
        session.setAttribute("username", userName);

        response.setContentType("text/html");
        PrintWriter out = response.getWriter();

        out.println("<!DOCTYPE html>");
        out.println("<html>");
        out.println("<head>");
        out.println("<meta charset='UTF-8'>");
        out.println("<title>Login Successful</title>");
        out.println("</head>");
        out.println("<body>");
        out.println("<h2>Login Successful</h2>");
        out.println("<p>Hello, " + userName + "!</p>");
        out.println("<p><a href='WelcomeServlet'>Go to Welcome    Page</a></p>");
        out.println("</body>");
        out.println("</html>");

        out.close();
    }
}
```

(c) Servlet – WelcomeServlet.java

```
import java.io.IOException;
import java.io.PrintWriter;
import javax.servlet.ServletException;
import javax.servlet.annotation.WebServlet;
import javax.servlet.http.HttpServlet;
import javax.servlet.http.HttpServletRequest;
import javax.servlet.http.HttpServletResponse;
import javax.servlet.http.HttpSession;

@WebServlet("/WelcomeServlet")
```

```
public class WelcomeServlet extends HttpServlet {

    private static final long serialVersionUID = 1L;

    @Override
    protected void doGet(HttpServletRequest request,
                         HttpServletResponse response)
            throws ServletException, IOException {

        response.setContentType("text/html");
        PrintWriter out = response.getWriter();

        HttpSession session = request.getSession(false); // Do not create new
        out.println("<!DOCTYPE html>");
        out.println("<html>");
        out.println("<head>");
        out.println("<meta charset='UTF-8'>");
        out.println("<title>Welcome</title>");
        out.println("</head>");
        out.println("<body>");

        if (session != null) {
            String userName = (String) session.getAttribute("username");
            if (userName != null) {
                out.println("<h2>Welcome, " + userName + "!</h2>");
                out.println("<p>Your Session ID: " + session.getId() + "</p>");
            } else {
                out.println("<h2>No user information found in session.</h2>");
            }
        } else {
            out.println("<h2>Session expired or not found. Please login again.</h2>");
        }

        out.println("</body>");
        out.println("</html>");

        out.close();
    }
}
```

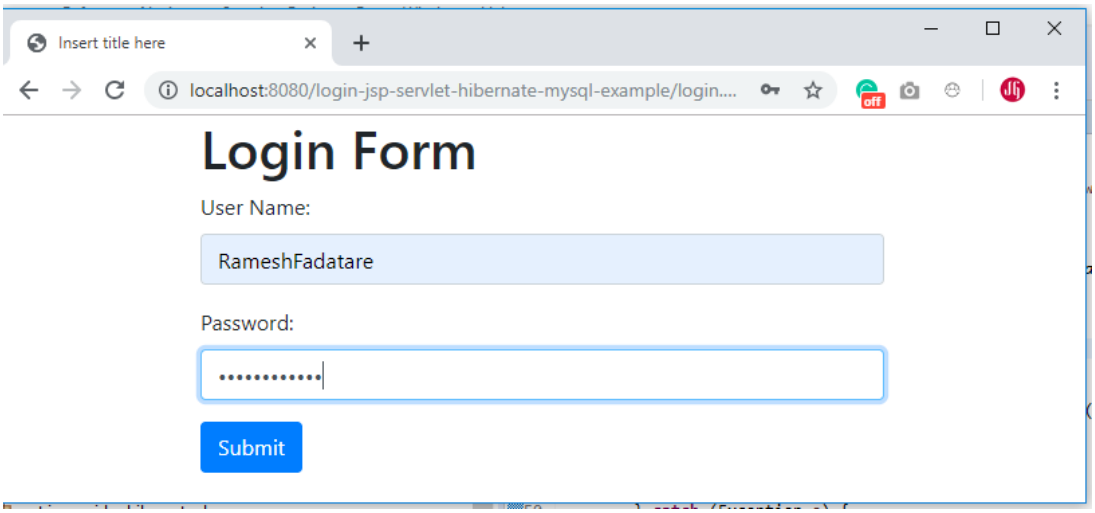
web.xml

```
<servlet>
    <servlet-name>LoginServlet</servlet-name>
```

```
    <servlet-class>LoginServlet</servlet-class>
</servlet>

<servlet-mapping>
    <servlet-name>LoginServlet</servlet-name>
    <url-pattern>/LoginServlet</url-pattern>
</servlet-mapping>

<servlet>
    <servlet-name>WelcomeServlet</servlet-name>
    <servlet-class>WelcomeServlet</servlet-class>
</servlet>

<servlet-mapping>
    <servlet-name>WelcomeServlet</servlet-name>
    <url-pattern>/WelcomeServlet</url-pattern>
</servlet-mapping>
```

Output

ogin.html



RESULT

Thus, session tracking was successfully implemented using HttpSession. The user name entered in the login page was stored as a session attribute and reused by another servlet to display a personalized welcome message and session details.

EXP 6(i) DATE: **CREATING A THREE–TIER APPLICATION USING JSP AND DATABASE**

AIM

To develop a three-tier web application using JSP and a database to conduct an online examination and display the student's score.

ALGORITHM

Step 1: Start the Apache Tomcat server and create a new Dynamic Web Project with JSP support in the IDE (e.g., Eclipse / NetBeans).
Step 2: Create the database college and the table questions in MySQL (or another RDBMS).
Step 3: Insert sample multiple-choice questions into the questions table with four options and a correct answer column.
Step 4: In the web project, create a JSP page exam.jsp to display all questions using data retrieved from the database.
Step 5: In exam.jsp, establish a JDBC connection using the MySQL driver, connection URL, username and password.
Step 6: Execute a SELECT query on the questions table and iterate through the ResultSet to display every question and its four options as radio buttons.
Step 7: Place all the questions and options inside an HTML <form> tag and set the form action to result.jsp and method to post.
Step 8: Create another JSP page result.jsp to process the submitted answers.
Step 9: In result.jsp, re-establish the JDBC connection and retrieve all questions again from the questions table.
Step 10: For each question, read the user's selected option from the request parameters and compare it with the correct_option field from the database.
Step 11: Maintain a counter for the total number of questions attempted and the number of correct answers.
Step 12: Display the total questions, correct answers and the final score using HTML in result.jsp.
Step 13: Save all files, deploy the project to Tomcat and run exam.jsp from the browser to test the online examination system.

DATABASE DESIGN (SQL SCRIPT)

```
CREATE DATABASE college;
USE college;

CREATE TABLE questions (
   qid INT PRIMARY KEY AUTO_INCREMENT,
   question      VARCHAR(255),
   optionA       VARCHAR(100),
```

```
    optionB      VARCHAR(100),
    optionC      VARCHAR(100),
    optionD      VARCHAR(100),
    correct_option CHAR(1)   -- 'A', 'B', 'C' or 'D'
);

INSERT INTO questions (question, optionA, optionB, optionC, optionD, correct_option)
VALUES
('Which language is used in JSP?', 'Java', 'C', 'C++', 'Python', 'A'),
('Which protocol is used on the web?', 'FTP', 'SMTP', 'HTTP', 'SSH', 'C'),
('Which method sends data securely?', 'GET', 'POST', 'HEAD', 'TRACE', 'B'),
('Which tag holds Java code in JSP?', '<%! %>', '<% %>', '<%@ %>', '<jsp:useBean>', 'B'),
('JDBC is used to connect?', 'HTML to DB', 'CSS to DB', 'Java to DB', 'XML to DB', 'C');
```

SOURCE CODE

(a) JSP Page – exam.jsp

```
<%@ page import="java.sql.*" %>
<!DOCTYPE html>
<html>
<head>
    <meta charset="UTF-8">
    <title>Online Examination</title>
</head>
<body>
    <h2>Online Examination</h2>

    <form action="result.jsp" method="post">
        <%
            Connection con = null;
            Statement stmt = null;
            ResultSet rs = null;

            String url  = "jdbc:mysql://localhost:3306/college";
            String user = "root";   // change as per DB
            String pass = "root";   // change as per DB

            try {
                Class.forName("com.mysql.cj.jdbc.Driver");
                con = DriverManager.getConnection(url, user, pass);
                stmt = con.createStatement();
                rs = stmt.executeQuery("SELECT * FROM questions");

                int qNo = 1;
```

```
            while (rs.next()) {
                int qid = rs.getInt("qid");
                String question = rs.getString("question");
                String optionA = rs.getString("optionA");
                String optionB = rs.getString("optionB");
                String optionC = rs.getString("optionC");
                String optionD = rs.getString("optionD");
        %>
                <b>Q<%= qNo %>. <%= question %></b><br>
                        <input type="radio" name="q<%= qid %>" value="A" required> <%=
            optionA                 %><br>
                <input type="radio" name="q<%= qid %>" value="B"> <%= optionB %><br>
                <input type="radio" name="q<%= qid %>" value="C"> <%= optionC %><br>
                <input type="radio" name="q<%= qid %>" value="D"> <%= optionD %><br>
                <br>
        <%
                qNo++;
            }
        } catch (Exception e) {
            out.println("Error: " + e.getMessage());
        } finally {
            try { if (rs != null) rs.close(); } catch (Exception e) {}
            try { if (stmt != null) stmt.close(); } catch (Exception e) {}
            try { if (con != null) con.close(); } catch (Exception e) {}
        }
        %>

        <input type="submit" value="Submit Test">
        <input type="reset" value="Clear">
    </form>
</body>
</html>
```

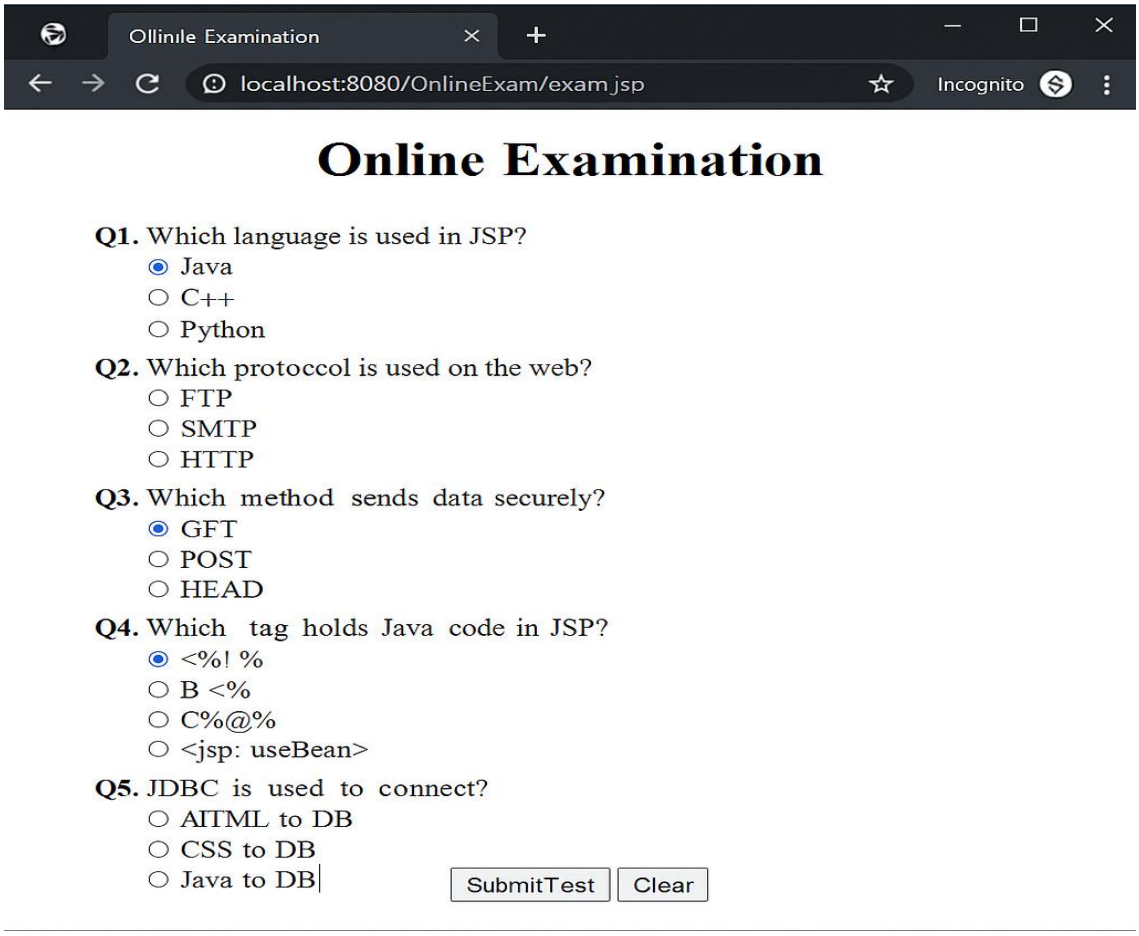
(b) JSP Page – result.jsp

```
<%@ page import="java.sql.*" %>
<!DOCTYPE html>
<html>
<head>
    <meta charset="UTF-8">
    <title>Exam Result</title>
</head>
<body>
    <h2>Online Examination Result</h2>
    <%
```

```
    Connection con = null;
    Statement stmt = null;
    ResultSet rs = null;

    String url  = "jdbc:mysql://localhost:3306/college";
    String user = "root";     // change as per DB
    String pass = "root";     // change as per DB

    int totalQuestions = 0;
    int correctAnswers = 0;

    try {
        Class.forName("com.mysql.cj.jdbc.Driver");
        con = DriverManager.getConnection(url, user, pass);
        stmt = con.createStatement();
        rs = stmt.executeQuery("SELECT * FROM questions");

        while (rs.next()) {
            int qid = rs.getInt("qid");
            String correctOption = rs.getString("correct_option");

            String paramName = "q" + qid;
            String selectedOption = request.getParameter(paramName);

            if (selectedOption != null) {
                totalQuestions++;
                if (selectedOption.equalsIgnoreCase(correctOption)) {
                    correctAnswers++;
                }
            }
        }
    } catch (Exception e) {
        out.println("Error: " + e.getMessage());
    } finally {
        try { if (rs != null) rs.close(); } catch (Exception e) {}
        try { if (stmt != null) stmt.close(); } catch (Exception e) {}
        try { if (con != null) con.close(); } catch (Exception e) {}
    }
%>

<p>Total Questions Attempted : <%= totalQuestions %></p>
<p>Correct Answers           : <%= correctAnswers %></p>
<p>Your Score                : <%= correctAnswers %> / <%= totalQuestions %></p>
```

```
</body>
</html>
```

Output



RESULT

Thus, a three-tier online examination system was successfully developed using JSP, JDBC and MySQL, and the student's score was computed and displayed based on the answers selected.

EXP 6(ii) CREATING A THREE–TIER APPLICATION USING JSP AND
DATE: DATABASE

AIM

To develop a three-tier web application using JSP and a database to display the mark list of a student, whose details are stored on a database server.

ALGORITHM

Step 1: Start the Apache Tomcat server and create a new Dynamic Web Project with JSP support.
Step 2: Create the database college and the table student_marks with fields for roll number, name, department and subject marks.
Step 3: Insert sample records for different students into the student_marks table.
Step 4: In the web project, create a JSP page searchStudent.jsp with a form to accept the student's roll number from the user.
Step 5: Set the form action to displayMark.jsp and the method to post.
Step 6: Create a JSP page displayMark.jsp to connect to the database using JDBC.
Step 7: In displayMark.jsp, read the roll number submitted in the form using request.getParameter("rollno").
Step 8: Prepare a parameterized SQL SELECT query using PreparedStatement to fetch details of the student whose roll number matches the input.
Step 9: Execute the query and store the result in a ResultSet.
Step 10: If a record is found, retrieve the student's name, department and marks in all subjects from the ResultSet.
Step 11: Compute the total marks and average marks in the JSP page.
Step 12: Derive the student's grade based on the average marks using simple conditional statements.
Step 13: Display the student details, subject marks, total, average and grade in an HTML table.
Step 14: If no record is found for the entered roll number, display an appropriate message to the user.
Step 15: Save all files, deploy the project and run searchStudent.jsp to verify correct retrieval and display of the student mark list.

DATABASE DESIGN (SQL SCRIPT)

```
USE college;

CREATE TABLE student_marks (
   rollno VARCHAR(20) PRIMARY KEY,
   name   VARCHAR(100),
   dept   VARCHAR(50),
```

```
    sub1   INT,
    sub2   INT,
    sub3   INT
);

INSERT INTO student_marks (rollno, name, dept, sub1, sub2, sub3) VALUES
('21CS001', 'Ananya', 'CSE', 85, 90, 88),
('21CS002', 'Ravi',   'CSE', 78, 82, 80),
('21CS003', 'Kiran',  'IT',  92, 89, 94);
```

SOURCE CODE

(a) JSP Page – searchStudent.jsp

```
<!DOCTYPE html>
<html>
<head>
    <meta charset="UTF-8">
    <title>Student Mark List - Search</title>
</head>
<body>
    <h2>Student Mark List</h2>
    <form action="displayMark.jsp" method="post">
        <table>
            <tr>
                <td>Enter Roll Number</td>
                <td><input type="text" name="rollno" required></td>
            </tr>
            <tr>
                <td colspan="2" style="text-align:center;">
                    <input type="submit" value="View Mark List">
                    <input type="reset" value="Clear">
                </td>
            </tr>
        </table>
    </form>
</body>
</html>
```

(b) JSP Page – displayMark.jsp

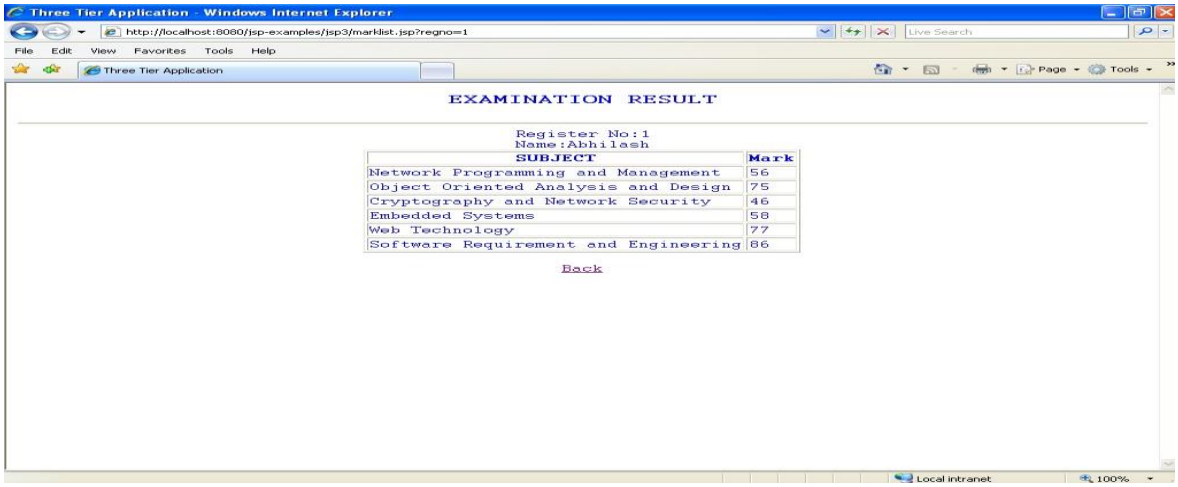
```
<%@ page import="java.sql.*" %>
<!DOCTYPE html>
<html>
<head>
    <meta charset="UTF-8">
    <title>Student Mark List</title>
```

```
</head>
<body>
   <h2>Student Mark List</h2>
   <%
      String roll = request.getParameter("rollno");

      Connection con = null;
      PreparedStatement ps = null;
      ResultSet rs = null;

      String url  = "jdbc:mysql://localhost:3306/college";
      String user = "root";    // change as per DB
      String pass = "root";    // change as per DB

      try {
         Class.forName("com.mysql.cj.jdbc.Driver");
         con = DriverManager.getConnection(url, user, pass);

         String query = "SELECT * FROM student_marks WHERE rollno = ?";
         ps = con.prepareStatement(query);
         ps.setString(1, roll);
         rs = ps.executeQuery();

         if (rs.next()) {
            String name = rs.getString("name");
            String dept = rs.getString("dept");
            int m1 = rs.getInt("sub1");
            int m2 = rs.getInt("sub2");
            int m3 = rs.getInt("sub3");

            int total = m1 + m2 + m3;
            float average = total / 3.0f;
            String grade;

            if (average >= 90)
               grade = "A+";
            else if (average >= 80)
               grade = "A";
            else if (average >= 70)
               grade = "B";
            else if (average >= 60)
               grade = "C";
            else
```

```
            grade = "D";
             %>
        <table border="1" cellpadding="5" cellspacing="0">
            <tr>
                <th>Roll Number</th>
                <td><%= roll %></td>
            </tr>
            <tr>
                <th>Name</th>
                <td><%= name %></td>
            </tr>
            <tr>
                <th>Department</th>
                <td><%= dept %></td>
            </tr>
            <tr>
                <th>Subject 1</th>
                <td><%= m1 %></td>
            </tr>
            <tr>
                <th>Subject 2</th>
                <td><%= m2 %></td>
            </tr>
            <tr>
                <th>Subject 3</th>
                <td><%= m3 %></td>
            </tr>
            <tr>
                <th>Total</th>
                <td><%= total %></td>
            </tr>
            <tr>
                <th>Average</th>
                <td><%= average %></td>
            </tr>
            <tr>
                <th>Grade</th>
                <td><%= grade %></td>
            </tr>
        </table>
<%
    } else {
%>
```

```
            <p>No record found for Roll Number: <b><%= roll %></b></p>
    <%
            }
        } catch (Exception e) {
            out.println("Error: " + e.getMessage());
        } finally {
            try { if (rs != null) rs.close(); } catch (Exception e) {}
            try { if (ps != null) ps.close(); } catch (Exception e) {}
            try { if (con != null) con.close(); } catch (Exception e) {}
        }
    %>
    <br>
    <a href="searchStudent.jsp">Search Another Student</a>
</body>
</html>
```

Output



```
Enter your DW Marks==>84
Enter your JAVA Marks==>93
Enter your SE Marks==>92
-----------------------------------------------------
Name = sourabh                          Roll_number7
-----------------------------------------------------
Subject                                        Marks
-----------------------------------------------------
Hindi                                           87
English                                         98
DW                                              84
JAVA                                            93
SE                                              92
-----------------------------------------------------
Division                 percent               90.0
first
```

RESULT

Thus, a three-tier student mark list application was successfully developed using JSP, JDBC and MySQL. The system retrieves student information from the database server and displays the complete mark list including total, average and grade for the entered roll number.

EXP 7 **XML Schema for Student Detail**
DATE:

AIM
To design an XML Schema (XSD) for representing student details and create a sample XML document that conforms to the schema.

ALGORITHM
Step 1: Identify the required information for each student, such as Register Number, Name, Department, Semester, Email and CGPA.
Step 2: Decide the root element name (e.g., students) and the child element name for each student record (e.g., student).
Step 3: For each student, define sub-elements such as regno, name, dept, semester, email and cgpa.
Step 4: Choose appropriate XML Schema data types for each element:

- Use xs:string for name, dept and email.
- Use xs:string or xs:token for regno.
- Use xs:positiveInteger for semester.
- Use xs:decimal for cgpa.

Step 5: Create a complex type (e.g., studentType) in the XSD that groups all student elements together in a sequence.
Step 6: Define the root element students as containing one or more student elements using maxOccurs="unbounded" and minOccurs="1".
Step 7: Apply simple constraints where required, such as:

- Restrict semester to a valid range (e.g., 1 to 8).
- Restrict cgpa to a valid range (e.g., 0.0 to 10.0).

Step 8: Save the schema as student.xsd.
Step 9: Create a sample XML document student.xml that uses the students root element and one or more student elements containing valid data.
Step 10: Reference the schema in the XML document using xsi:noNamespaceSchemaLocation.
Step 11: Validate the XML document against the XML Schema using an XML editor or any XML validation tool.
Step 12: Ensure that the XML document is valid and correctly follows all the rules specified in the schema.

SOURCE CODE

(a) XML Schema – student.xsd

```
<?xml version="1.0" encoding="UTF-8"?>
```

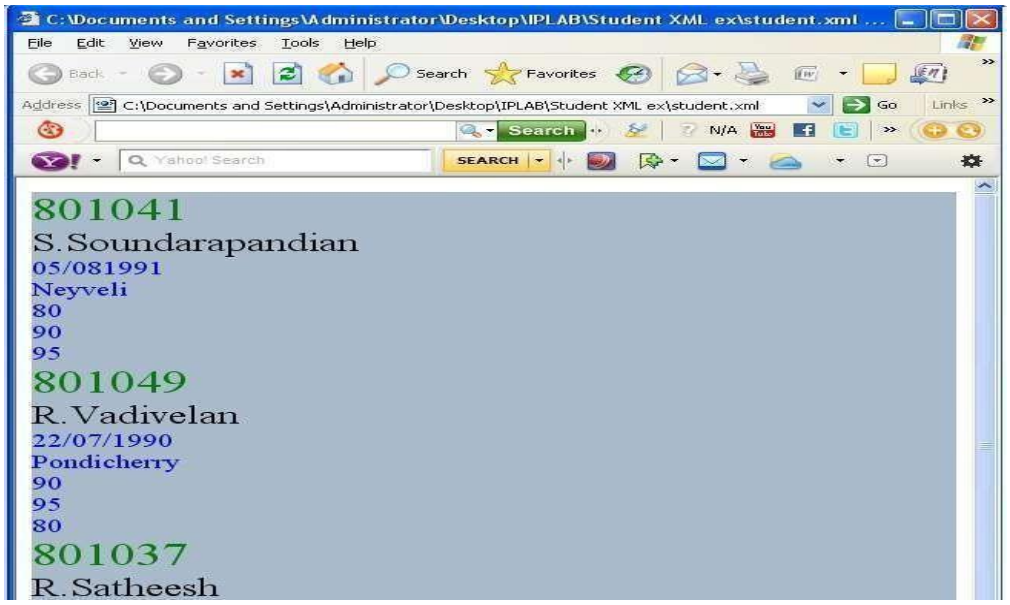
```
<xs:schema xmlns:xs="http://www.w3.org/2001/XMLSchema">

  <!-- Complex type for a single student -->
  <xs:complexType name="studentType">
    <xs:sequence>
      <xs:element name="regno" type="xs:string"/>
      <xs:element name="name" type="xs:string"/>
      <xs:element name="dept" type="xs:string"/>
      <xs:element name="semester">
        <xs:simpleType>
          <xs:restriction base="xs:positiveInteger">
            <xs:minInclusive value="1"/>
            <xs:maxInclusive value="8"/>
          </xs:restriction>
        </xs:simpleType>
      </xs:element>
      <xs:element name="email" type="xs:string"/>
      <xs:element name="cgpa">
        <xs:simpleType>
          <xs:restriction base="xs:decimal">
            <xs:minInclusive value="0.0"/>
            <xs:maxInclusive value="10.0"/>
          </xs:restriction>
        </xs:simpleType>
      </xs:element>
    </xs:sequence>
  </xs:complexType>

  <!-- Root element containing a list of students -->
  <xs:element name="students">
    <xs:complexType>
      <xs:sequence>
        <xs:element name="student" type="studentType"
                    minOccurs="1" maxOccurs="unbounded"/>
      </xs:sequence>
    </xs:complexType>
  </xs:element>

</xs:schema>
```

(b) Sample XML Document – student.xml

```
<?xml version="1.0" encoding="UTF-8"?>
<students xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
```

```
        xsi:noNamespaceSchemaLocation="student.xsd">

    <student>
        <regno>21CS001</regno>
        <name>Ananya</name>
        <dept>CSE</dept>
        <semester>5</semester>
        <email>ananya@college.edu</email>
        <cgpa>9.1</cgpa>
    </student>

    <student>
        <regno>21CS002</regno>
        <name>Ravi</name>
        <dept>CSE</dept>
        <semester>5</semester>
        <email>ravi@college.edu</email>
        <cgpa>8.4</cgpa>
    </student>

    <student>
        <regno>21CS003</regno>
        <name>Kiran</name>
        <dept>IT</dept>
        <semester>3</semester>
        <email>kiran@college.edu</email>
        <cgpa>8.9</cgpa>
    </student>

</students>
```

Output:



RESULT

Thus, an XML Schema for student details was successfully designed, and a sample XML document was created and validated against the schema, ensuring that all student records conform to the defined structure and constraints.